

Table of Contents

[Preface](#)

[Part 1: Getting Started with Full-Stack Development](#)

[1](#)

[Preparing for Full-Stack Development](#)

[Technical requirements](#)

[Motivation to become a full-stack developer](#)

[What is new in this release of Full-Stack React Projects?](#)

[Getting the most out of this book](#)

[Setting up the development environment](#)

[Installing VS Code and extensions](#)

[Setting up a project with Vite](#)

[Setting up ESLint and Prettier to enforce best practices and code style](#)

Setting up Husky to make sure we commit proper code

Summary

2

Getting to Know Node.js and MongoDB

Technical requirements

Writing and running scripts with Node.js

Similarities and differences between JavaScript in the browser and in Node.js

Creating our first Node.js script

Handling files in Node.js

Concurrency with JavaScript in the browser and Node.js

Creating our first web server

Extending the web server to serve our JSON file

Introducing Docker, a platform for containers

The Docker platform

Installing Docker

Creating a container

Accessing Docker via VS Code

[Introducing MongoDB, a document database](#)

[Setting up a MongoDB server](#)

[Running commands directly on the database](#)

[Accessing the database via VS Code](#)

[Accessing the MongoDB database via Node.js](#)

[Summary](#)

[Part 2: Building and Deploying Our First Full-Stack Application with a REST API](#)

[3](#)

[Implementing a Backend Using Express, Mongoose ODM, and Jest](#)

[Technical requirements](#)

[Designing a backend service](#)

[Creating the folder structure for our backend service](#)

[Creating database schemas using Mongoose](#)

[Defining a model for blog posts](#)

[Using the blog_post model](#)

[Defining creation and last update dates in the blog_post](#)

[Developing and testing service functions](#)

Setting up the test environment

Writing our first service function: createPost

Defining test cases for the createPost service function

Defining a function to list posts

Defining test cases for list posts

Defining the get single post, update and delete post functions

Using the Jest VS Code extension

Providing a REST API using Express

Defining our API routes

Setting up Express

Using dotenv for setting environment variables

Using nodemon for easier development

Creating our API routes with Express

Summary

4

Integrating a Frontend Using React and TanStack Query

Technical requirements

Principles of React

Setting up a full-stack React project

Creating the user interface for our application

Component structure

Implementing static React components

Integrating the backend service using TanStack Query

Setting up TanStack Query for React

Fetching blog posts

Implementing filters and sorting

Creating new posts

Summary

5

Deploying the Application with Docker and CI/CD

Technical requirements

Creating Docker images

Creating the backend Dockerfile

Creating a .dockerignore file

Building the Docker image

[Creating and running a container from our image](#)

[Creating the frontend Dockerfile](#)

[Creating the .dockerignore file for the frontend](#)

[Building the frontend Docker image](#)

[Creating and running the frontend container](#)

[Managing multiple images using Docker Compose](#)

[Cleaning up unused containers](#)

[Deploying our full-stack application to the cloud](#)

[Creating a MongoDB Atlas database](#)

[Creating an account on Google Cloud](#)

[Deploying our Docker images to a Docker registry](#)

[Deploying the backend Docker image to Cloud Run](#)

[Deploying the frontend Docker image to Cloud Run](#)

[Configuring CI to automate testing](#)

[Adding CI for the frontend](#)

[Adding CI for the backend](#)

[Configuring CD to automate the deployment](#)

[Getting Docker Hub credentials](#)

[Getting Google Cloud credentials](#)

Defining the deployment workflow

Summary

Part 3: Practicing Development of Full-Stack Web Applications

6

Adding Authentication with JWT

Technical requirements

What is JWT?

JWT header

JWT payload

JWT signature

Creating a JWT

Using JWT

Storing JWT

Implementing login, signup, and authenticated routes in the backend using JWTs

Creating the user model

Creating the signup service

Creating the signup route

Creating the login service

[Creating the login route](#)

[Defining authenticated routes](#)

[Accessing the currently logged-in user](#)

[Integrating login and signup in the frontend using React Router and JWT](#)

[Using React Router to implement multiple routes](#)

[Creating the signup page](#)

[Linking to other routes using the Link component](#)

[Creating the login page and storing the JWT](#)

[Using the stored JWT and implementing a simple logout](#)

[Fetching the usernames](#)

[Sending the JWT header when creating posts](#)

[Advanced token handling](#)

[Summary](#)

[7](#)

[Improving the Load Time Using Server-Side Rendering](#)

[Technical requirements](#)

[Benchmarking the load time of our application](#)

Rendering React components on the server

Setting up the server

Defining the server-side entry point

Defining the client-side entry point

Updating index.html and package.json

Making React Router work with server-side rendering

Server-side data fetching

Using initial data

Using hydration

Advanced server-side rendering

Summary

8

Making Sure Customers Find You with Search Engine Optimization

Technical requirements

Optimizing an application for search engines

Creating a robots.txt file

Creating separate pages for posts

Creating meaningful URLs (slugs)

[Adding dynamic titles](#)

[Adding other meta tags](#)

[Creating a sitemap](#)

[Improving social media embeds](#)

[Open Graph meta tags](#)

[Using the OG article meta tags](#)

[Summary](#)

[9](#)

[Implementing End-to-End Tests Using Playwright](#)

[Technical requirements](#)

[Setting up Playwright for end-to-end testing](#)

[Installing Playwright](#)

[Preparing the backend for end-to-end testing](#)

[Writing and running end-to-end tests](#)

[Using the VS Code extension](#)

[Reusable test setups using fixtures](#)

[Overview of built-in fixtures](#)

[Writing our own fixture](#)

[Using custom fixtures](#)

[Viewing test reports and running in CI](#)

[Viewing an HTML report](#)

[Running Playwright tests in CI](#)

[Summary](#)

[10](#)

[Aggregating and Visualizing Statistics
Using MongoDB and Victory](#)

[Technical requirements](#)

[Collecting and simulating events](#)

[Creating the event model](#)

[Defining a service function and route to track
events](#)

[Collecting events on the frontend](#)

[Simulating events](#)

[Aggregating data with MongoDB](#)

[Getting the total number of views per post](#)

[Getting the number of daily views per post](#)

[Calculating the average session duration](#)

[Implementing data aggregation in the backend](#)

Defining aggregation service functions

Defining the routes

Integrating and visualizing data on the frontend using Victory

Integrating the aggregation API

Visualizing data using Victory

Summary

11

Building a Backend with a GraphQL API

Technical requirements

What is GraphQL?

Mutations

Implementing a GraphQL API in a backend

Implementing fields that query posts

Defining the Post type

Defining the User type

Trying out deeply nested queries

Implementing input types

Implementing GraphQL authentication and mutations

[Adding authentication to GraphQL](#)

[Implementing mutations](#)

[Using mutations](#)

[Overview of advanced GraphQL concepts](#)

[Fragments](#)

[Introspection](#)

[Summary](#)

[12](#)

[Interfacing with GraphQL on the Frontend Using Apollo Client](#)

[Technical requirements](#)

[Setting up Apollo Client and making our first query](#)

[Querying posts from the frontend using GraphQL](#)

[Resolving author usernames in a single query](#)

[Using variables in GraphQL queries](#)

[Using fragments to reuse parts of queries](#)

[Using mutations on the frontend](#)

[Migrating login to GraphQL](#)

[Migrating create post to GraphQL](#)

[Summary](#)

[Part 4: Exploring an Event-Based Full-Stack Architecture](#)

[13](#)

[Building an Event-Based Backend Using Express and Socket.IO](#)

[Technical requirements](#)

[What are event-based applications?](#)

[What are WebSockets?](#)

[What is Socket.IO?](#)

[Connecting to Socket.IO](#)

[Emitting and receiving events](#)

[Setting up Socket.IO](#)

[Setting up a simple Socket.IO client](#)

[Creating a backend for a chat app using Socket.IO](#)

[Emitting events to send chat messages from the client to the server](#)

[Broadcasting chat messages from the server to all clients](#)

[Joining rooms to send messages in](#)

Using acknowledgments to get information about a user

Adding authentication by integrating JWT with Socket.IO

Summary

14

Creating a Frontend to Consume and Send Events

Technical requirements

Integrating the Socket.IO client with React

Cleaning up the project

Creating a Socket.IO context

Hooking up the context and displaying the status

Disconnecting socket on logout

Implementing chat functionality

Implementing the chat components

Implementing a useChat hook

Implementing the ChatRoom component

Implementing chat commands with acknowledgments

Summary

15

Adding Persistence to Socket.IO Using MongoDB

Technical requirements

Storing and replaying messages using MongoDB

Creating the Mongoose schema

Creating the service functions

Storing and replaying messages

Visually distinguishing replayed messages

Refactoring the app to be more extensible

Defining service functions

Refactoring the Socket.IO server to use the service functions

Refactoring the client-side code

Implementing commands to join and switch rooms

Summary

Part 5: Advancing to Enterprise-Ready Full-Stack Applications

16

Getting Started with Next.js

Technical requirements

What is Next.js?

Setting up Next.js

Introducing the App Router

Defining the folder structure

Creating static components and pages

Defining components

Defining pages

Adding links between pages

Summary

17

Introducing React Server Components

Technical requirements

What are RSCs?

Adding a data layer to our Next.js app

Setting up the database connection

Creating the database models

Defining data layer functions

Using RSCs to fetch data from the database

Fetching a list of posts

Fetching a single post

Using Server Actions to sign up, log in, and create new posts

Implementing the signup page

Implementing the login page and JWT handling

Checking if the user is logged in

Implementing logout

Implementing post creation

Summary

18

Advanced Next.js Concepts and Optimizations

Technical requirements

Defining API routes in Next.js

Creating an API route for listing blog posts

Caching in Next.js

Exploring static rendering in API routes

Making the route dynamic

Caching functions in the data layer

[Revalidating the cache via Server Actions](#)

[Revalidating the cache via a Webhook](#)

[Revalidating the cache periodically](#)

[Opting out of caching](#)

[SEO with Next.js](#)

[Adding dynamic titles and meta tags](#)

[Creating a robots.txt file](#)

[Creating meaningful URLs \(slugs\)](#)

[Creating a sitemap](#)

[Optimized image and font loading in Next.js](#)

[The Font component](#)

[The Image component](#)

[Summary](#)

[19](#)

[Deploying a Next.js App](#)

[Technical requirements](#)

[Deploying a Next.js app with Vercel](#)

[Setting environment variables in Vercel](#)

[Creating a custom deployment setup for Next.js apps](#)

[Summary](#)

[20](#)

[Diving Deeper into Full-Stack Development](#)

[Overview of other full-stack frameworks](#)

[Next.js](#)

[Remix](#)

[Gatsby](#)

[Overview of UI libraries](#)

[Material UI \(MUI\)](#)

[Tailwind CSS](#)

[React Aria](#)

[NextUI](#)

[Overview of advanced state management solutions](#)

[Recoil](#)

[Jotai](#)

[Redux](#)

[MobX](#)

[xstate](#)

[Zustand](#)

[Pointers on maintaining large-scale projects](#)

[Using TypeScript](#)

[Setting up a Monorepo](#)

[Optimizing the bundle size](#)

[Summary](#)

[Index](#)

[Other Books You May Enjoy](#)